



Orleans.GpuBridge

GPU-Native Distributed Computing for Orleans

Sub-Microsecond Actor Messaging on GPU Hardware

.NET 9 • Ring Kernels • Temporal Alignment • DotCompute Backend

Executive Overview

Version 0.2.1

A paradigm shift from CPU-based actor systems to actors that live permanently on the GPU — achieving 100–500 ns message latency and 2 M messages/second throughput.

Built with .NET 9 & C# • 11 modular packages • 1,200+ tests

Open-source (Apache-2.0) • Commercial license available

Contact: michael.ivertowski@ch.ey.com

Contents

1	Executive Summary	2
1.1	At a Glance	2
2	The GPU-Native Paradigm	2
2.1	Traditional vs. GPU-Native Actors	2
2.2	Performance Comparison	3
3	Core Technology	3
3.1	Ring Kernel Infrastructure	3
3.2	Temporal Alignment	3
3.3	GPU-to-GPU Direct Messaging	4
4	Architecture	4
4.1	Package Structure	4
4.2	Key Packages	5
5	Advanced Features	6
5.1	Hypergraph Actors	6
5.2	Adaptive Placement & Load Balancing	6
5.3	Resilience Patterns	6
6	GPU Platform Support	7
6.1	Backend Matrix	7
6.2	GPU Hardware Tiers	7
7	Deployment Models	7
8	Use Cases	8
9	Quality & Maturity	9
9.1	Test Coverage	9
9.2	Implementation Status	9
10	Getting Started	10
10.1	Installation	10
10.2	Basic GPU Grain	10
10.3	Build & Test	10
11	Links & Resources	11

1 Executive Summary

Orleans.GpuBridge is a .NET 9 library that enables GPU-native distributed computing for Microsoft Orleans. Rather than treating the GPU as an accelerator to which work is occasionally dispatched, it introduces a fundamentally new model: *actors that reside permanently in GPU memory*, communicating through lock-free ring-buffer message queues at hardware speed.

The result is a 20–200× improvement over traditional CPU-based actor systems, with message latencies of 100–500 ns on datacenter GPUs and throughput exceeding 2 M messages/second per actor.

Why Orleans.GpuBridge?

- **GPU-native actors** — actors live permanently in GPU memory; zero kernel-launch overhead eliminates the dominant cost of traditional GPU offloading.
- **Ring kernels** — persistent GPU kernels running infinite dispatch loops process messages without repeated launch cycles.
- **Temporal alignment** — Hybrid Logical Clocks and Vector Clocks execute *on the GPU*, providing causal ordering at hardware speed.
- **Familiar .NET patterns** — standard `async/await` C# programming model with automatic CPU fallback when no GPU is available.
- **Production-grade resilience** — Polly v8 integration with circuit breakers, rate limiting, retry policies, and comprehensive health checks.
- **Cross-platform GPU support** — CUDA, OpenCL, DirectCompute, and Metal through the DotCompute abstraction layer.

1.1 At a Glance

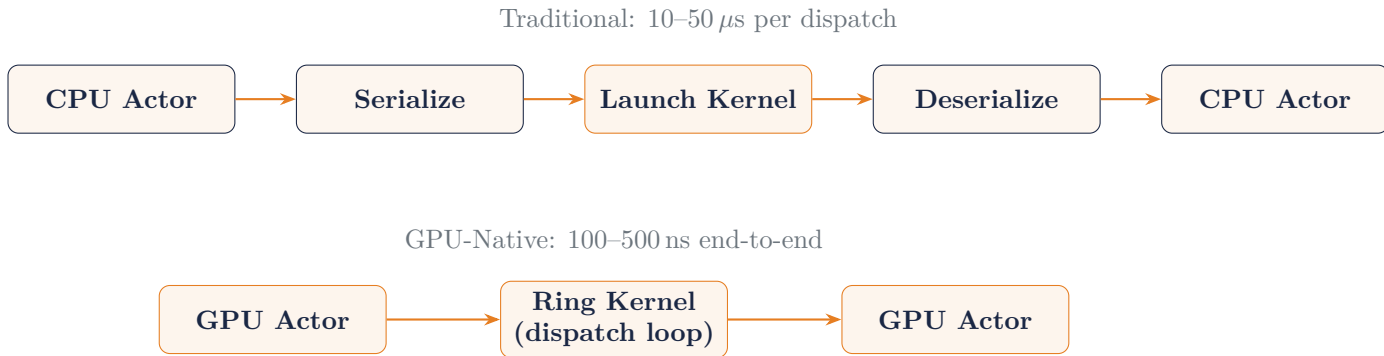


2 The GPU-Native Paradigm

2.1 Traditional vs. GPU-Native Actors

The conventional approach treats the GPU as a coprocessor: CPU actors serialize data, launch a kernel, wait for completion, and deserialize results. Each dispatch incurs 10–50 μ s of launch overhead.

Orleans.GpuBridge eliminates this entirely. Actors are allocated in GPU memory and communicate through lock-free ring-buffer queues. A persistent *ring kernel* runs an infinite dispatch loop, routing messages to actors without ever returning to the CPU.



2.2 Performance Comparison

Metric	CPU Actors	GPU Offload	GPU-Native
Message Latency	10–100 μ s	10–50 μ s	100–500 ns
Throughput	15 K msg/s	50 K msg/s	2 M msg/s
Memory Bandwidth	200 GB/s	500 GB/s	1,935 GB/s
Kernel Overhead	N/A	10–50 μ s	0 (persistent)
Improvement	—	2–5 $\times$	20–200$\times$

3 Core Technology

3.1 Ring Kernel Infrastructure

Ring kernels are persistent GPU kernels that never terminate. Each kernel runs an infinite dispatch loop, polling a lock-free message queue and routing messages to GPU-resident actors. This architecture delivers three key properties:

1. **Zero launch overhead** — the kernel is always running; no CUDA `cudaLaunchKernel` calls are needed per message.
2. **GPU-resident state** — actor state lives in GPU global memory, eliminating PCIe transfer costs.
3. **Hardware-speed messaging** — messages traverse GPU shared memory at DRAM bandwidth (>1 TB/s on modern GPUs).

3.2 Temporal Alignment

Distributed actor systems require causal ordering of events. Orleans.GpuBridge implements two complementary clock mechanisms *directly on the GPU*:

Mechanism	Description
Hybrid Logical Clock	Combines physical wall-clock time with a logical counter to provide globally ordered timestamps without synchronization barriers.
Vector Clocks	Per-actor logical clocks that capture causal “happens-before” relationships, enabling precise conflict detection in concurrent GPU operations.
Latency Compensator	Adaptive calibration using exponentially weighted moving averages to compensate for inter-node clock skew in multi-silo deployments.

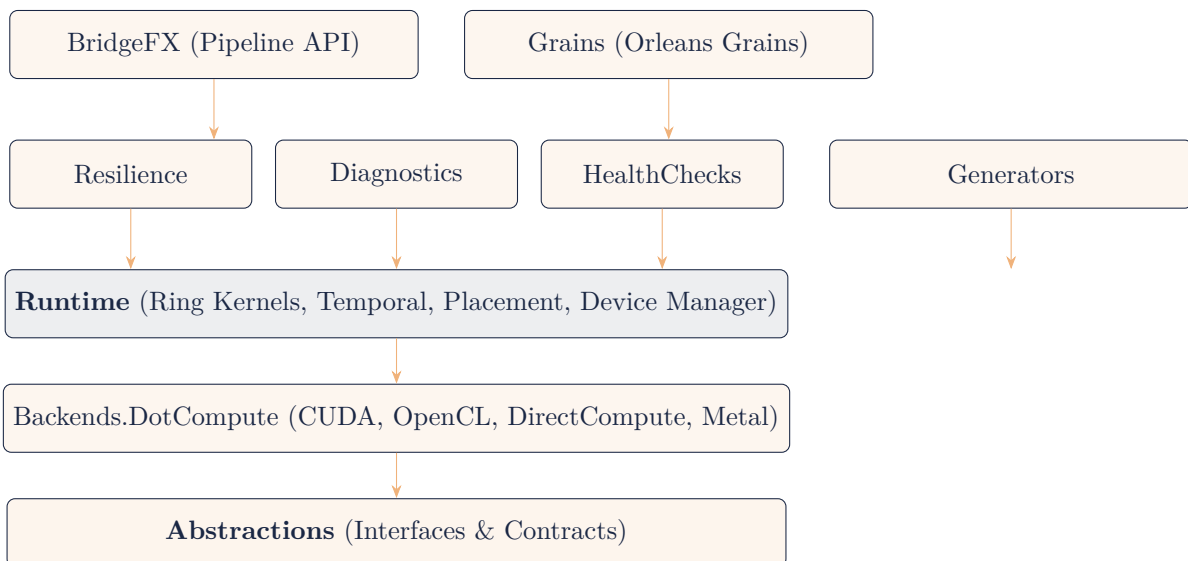
3.3 GPU-to-GPU Direct Messaging

Actors on different GPUs communicate through a peer-to-peer messaging layer that avoids CPU round-trips. When NVLink or peer-to-peer PCIe access is available, messages bypass the CPU entirely. On systems without P2P support, the framework automatically falls back to CPU-mediated transfers with no API changes required.

4 Architecture

4.1 Package Structure

Orleans.GpuBridge is organized as eleven modular NuGet packages, each with a clear responsibility boundary:



4.2 Key Packages

Package	Responsibility
Abstractions	Core interfaces: <code>IGpuDevice</code> , <code>IGpuKernel</code> , <code>IGpuBuffer</code> , <code>IRingKernelRuntime</code> , temporal contracts.
Runtime	Ring kernel management, device discovery, queue-depth placement, adaptive load balancing, HLC/Vector Clock implementation.
Grains	Orleans grain base classes: <code>GpuGrainBase</code> , <code>RingKernelGrainBase</code> with lifecycle integration.
BridgeFX	High-level fluent API for GPU pipeline construction with <code>async/await</code> support.
Backends.DotCompute	Cross-platform GPU backend via DotCompute — CUDA, OpenCL, DirectCompute, and Metal.
Resilience	Polly v8 policies: circuit breaker, retry, timeout, rate limiting, and bulkhead isolation.
Diagnostics	OpenTelemetry metrics, GPU memory telemetry, structured logging.
HealthChecks	ASP.NET Core health checks for GPU device status and kernel liveness.
Generators	Compile-time source generators for GPU actor boilerplate.

5 Advanced Features

5.1 Hypergraph Actors

Traditional actor graphs model pairwise relationships. Orleans.GpuBridge introduces *hypergraph actors* that represent multi-way relationships — a single hyperedge can connect three or more actors simultaneously.

Hypergraph Advantages

- **Pattern matching** — GPU-accelerated subgraph isomorphism at 10–500× the speed of CPU implementations.
- **Multi-party transactions** — model complex business events involving multiple entities without decomposing into pairwise edges.
- **Temporal hyperedges** — each relationship carries HLC timestamps for causal ordering and temporal queries.
- **Dynamic topology** — actors can join/leave hyperedges at runtime without global coordination.

5.2 Adaptive Placement & Load Balancing

Orleans.GpuBridge extends Orleans' placement model with GPU-aware strategies:

Strategy	Behaviour
Queue-Depth Placement	Routes grain activations to the GPU with the shortest message queue, minimizing tail latency.
Adaptive Load Balancer	Continuously monitors GPU utilization, memory pressure, and thermal state; rebalances actors across devices when thresholds are exceeded.
Memory-Aware Eviction	When GPU memory is constrained, cold actors are transparently migrated to CPU with automatic re-promotion on next access.

5.3 Resilience Patterns

Built on Polly v8, the resilience layer provides production-grade fault tolerance:

- **Circuit breaker** — halts GPU dispatch after configurable failure thresholds; auto-recovers after a cooling period.
- **Retry with backoff** — exponential and jittered retry for transient GPU errors (driver timeouts, memory allocation failures).
- **Rate limiting** — token-bucket rate limiter prevents GPU overload during burst traffic.
- **Timeout management** — per-kernel and per-pipeline timeouts with graceful cancellation via `CancellationToken`.
- **Bulkhead isolation** — limits concurrent GPU operations to prevent resource starvation.

6 GPU Platform Support

6.1 Backend Matrix

Orleans.GpuBridge supports five GPU compute backends through the DotCompute abstraction layer. A unified API surface ensures application code is backend-agnostic.

Backend	Windows	Linux	macOS	Notes
CUDA	✓	✓	—	NVIDIA GPUs; full ring kernel support
OpenCL	✓	✓	✓	Multi-vendor; broadest hardware reach
DirectCompute	✓	—	—	DirectX 11+ integrated workloads
Metal	—	—	✓	Apple Silicon native compute
Vulkan		<i>planned</i>		Cross-vendor next-gen API

6.2 GPU Hardware Tiers

The ring kernel execution model adapts to the coherence capabilities of the target GPU:

Tier	Mode	Latency	Hardware Examples
Full Coherence	Persistent kernel	100–500 ns	A100, H100, Grace Hopper
Partial Coherence	EventDriven	1–10 ms	RTX 2000/3000/4000 series
Limited	EventDriven	~5 s	WSL2 environments; development only

On hardware without GPU coherence, the framework transparently falls back to EventDriven mode — the same API, different dispatch strategy, no code changes required.

7 Deployment Models

Orleans.GpuBridge supports two deployment models that coexist in the same Orleans silo. Developers choose per-grain which model to use, and the framework handles resource allocation, fallback, and lifecycle management.

Model	Throughput	Best For
GPU-Offload	15 K msg/s	Batch processing, infrequent GPU usage (<10 calls/sec). CPU actors dispatch kernels to GPU on demand.
GPU-Native	2 M msg/s	High-frequency messaging (>1 K msg/s), real-time analytics. Actors live permanently in GPU memory with ring kernels.

8 Use Cases

Primary Use Cases

- ▶ **High-Frequency Trading** — Order matching and risk computation at $<10\mu s$ with GPU-resident market data and temporal causal ordering.
- ▶ **Real-Time Fraud Detection** — Hypergraph pattern matching across transaction networks with sub-millisecond anomaly detection.
- ▶ **Digital Twins** — Physics-accurate simulation with GPU-native actors representing sensors, actuators, and control systems at hardware speed.
- ▶ **Process Intelligence** — Object-Centric Process Mining with $640\times$ speedup for process discovery, conformance checking at $450\mu s$.
- ▶ **Stream Processing** — GPU-accelerated event stream analytics with $<100\mu s$ end-to-end latency for IoT, telemetry, and log analysis.
- ▶ **Scientific Computing** — Molecular dynamics, particle systems, and genome alignment using the familiar Orleans actor model with GPU acceleration.
- ▶ **Graph Analytics** — Temporal hypergraph queries, centrality computation, and community detection at GPU memory bandwidth.

9 Quality & Maturity

9.1 Test Coverage

The project includes **1,153 automated tests** across 11 test suites, with **1,133 passing** and 18 skipped (all with documented reasons: GPU hardware requirements, full Orleans silo infrastructure, or lock-free data structure work-in-progress).

Test Project	Total	Test Project	Total
Abstractions.Tests	242	Hardware.Tests	37
Runtime.Tests	206	Backends.DotCompute.Tests	56
Temporal.Tests	292	RingKernelTests	92
Grains.Tests	98	Performance.Tests	20
Generators.Tests	22	Integration.Tests	35
		Resilience.Tests	53

9.2 Implementation Status

Feature	Details
✓ Ring kernel infrastructure	Persistent dispatch loops, message routing
✓ GPU-resident message queues	Lock-free ring buffers in GPU memory
✓ Temporal alignment	HLC & Vector Clocks on GPU
✓ Hypergraph actors	Multi-way relationships, pattern matching
✓ Queue-depth placement	GPU-aware grain activation routing
✓ Adaptive load balancing	Dynamic rebalancing across devices
✓ Resilience (Polly v8)	Circuit breaker, retry, rate limiting
✓ GPU Direct Messaging	P2P with automatic fallback
✓ GPU Memory Telemetry	Per-grain OpenTelemetry metrics
✓ DotCompute backend	EventDriven mode production-ready
○ Persistent ring kernel mode	Requires full GPU coherence hardware
○ Vulkan compute backend	Cross-vendor next-gen API
○ GPUDirect Storage	NVMe-to-GPU direct path

10 Getting Started

10.1 Installation

```
# Add core packages
dotnet add package Orleans.GpuBridge.Abstractions
dotnet add package Orleans.GpuBridge.Runtime
dotnet add package Orleans.GpuBridge.Grains

# Add resilience and diagnostics
dotnet add package Orleans.GpuBridge.Resilience
dotnet add package Orleans.GpuBridge.Diagnostics
dotnet add package Orleans.GpuBridge.HealthChecks
```

10.2 Basic GPU Grain

```
public class MyGpuGrain : GpuGrainBase, IMyGpuGrain
{
    public async Task<float[]> ComputeAsync(float[] input)
    {
        using var buffer = await GpuDevice.AllocateAsync(input);
        await GpuDevice.DispatchAsync("my_kernel", buffer);
        return await buffer.ReadAsync();
    }
}
```

10.3 Build & Test

```
# Build the solution
dotnet build

# Run all tests
dotnet test

# Build release with NuGet packages
dotnet pack -c Release
```

11 Links & Resources

Repository and Ecosystem

Repository	https://github.com/mivertowski/Orleans.GpuBridge.Core
Documentation	docs/ directory — architecture overviews, API reference, article series on GPU-native actors, hypergraph actors, and temporal correctness.
NuGet	Orleans.GpuBridge.* packages (Abstractions, Runtime, Grains, BridgeFX, Resilience, Diagnostics, HealthChecks, Generators)
License	Apache-2.0 (open-source); commercial license available
Microsoft Orleans	Distributed actor framework — virtual actors, cluster management, and grain lifecycle; the foundation this library extends.
DotCompute	.NET-native GPU compute abstraction unifying CUDA, OpenCL, DirectCompute, and Metal under a single API.
DataSynth	Enterprise synthetic data platform for GPU-accelerated analytics.
AssureTwin	Native audit intelligence platform using GpuBridge for GPU-accelerated graph analytics and process mining.